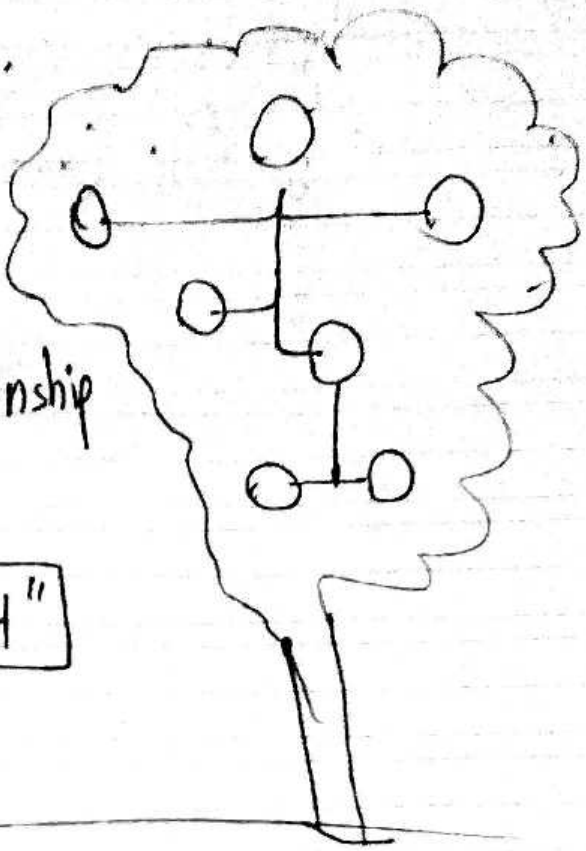
DSA ① [2]

Data structure: a named location that can be used to store and organize data.

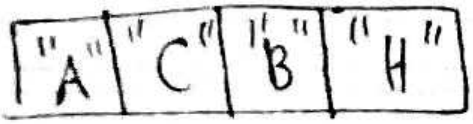
examples:

① family tree

- heirarchy of family relathionship



② array



Algorithm: is a collection of steps to solve a problem

example: linear search

one by one

---

why DSA?

1. you'll write code that is both time and memory efficient.

2. commonly asked (Q) involves DS & A in interviews

missing:

(1) ~~priority Queue~~ ✓

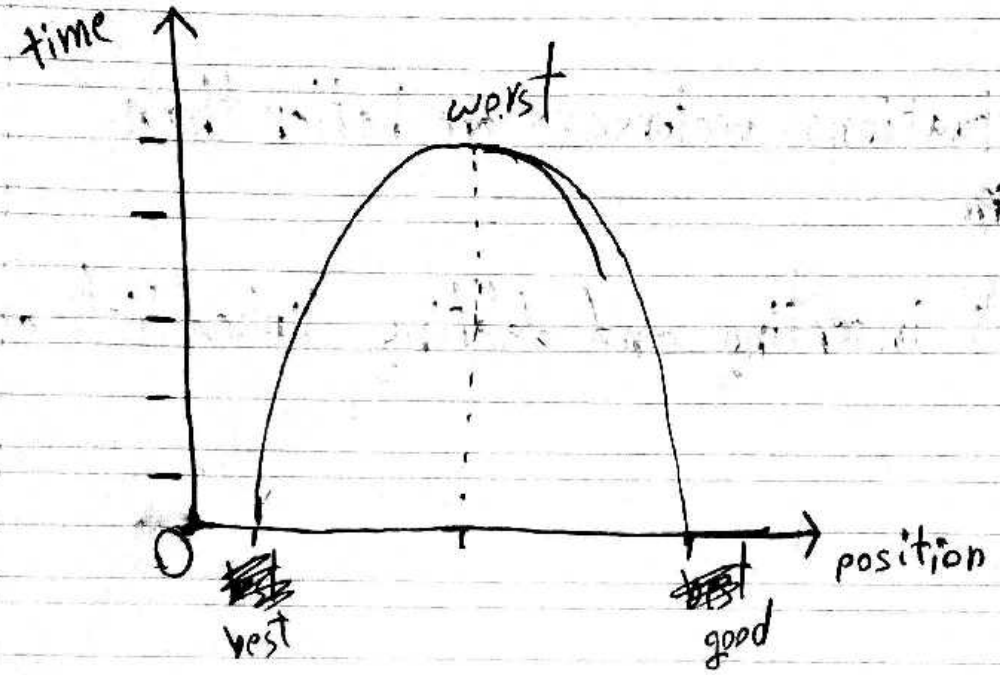
(2) list get(); (access method).

vectors faster in access than lists

6400 ns 11800 ns

	vector	list
0	6400	11800
500000	6900	7537100
999999	17800	63700

Linked List get (access)



remove

Linked Lists are better than vectors in removing & (caused of shifting)

	0	500000	999999	pos
Linked List	17200	7040600	23300	ns
Vector	2241200	1629700	18600	ns

conclusion:

in most situations vectors<T> are better than lists<T>

for a lot of inserting and deleting linked lists are better

Date / / (5)

very very basics of: Big O notation

"How code slows as data grows"

1. Describes the performance of an algorithm as the amount of ~~data~~ data increases

2. this notation is: Machine independent (# of steps to completion) number

3. ignore smaller operations: $O(n+1) \Rightarrow O(n)$

examples:

$O(1)$

$O(n)$

$O(\log n)$

$O(n^2)$

n = amount of data (it's a variable like x)

(1)

O(n) linear time

```
int addUp(int n){
  int sum = 0;
  for(int i = 0; i <= n; i++){
    sum += 1;
  }
  return sum;
}
```

~ 1000000 steps n = 1000000

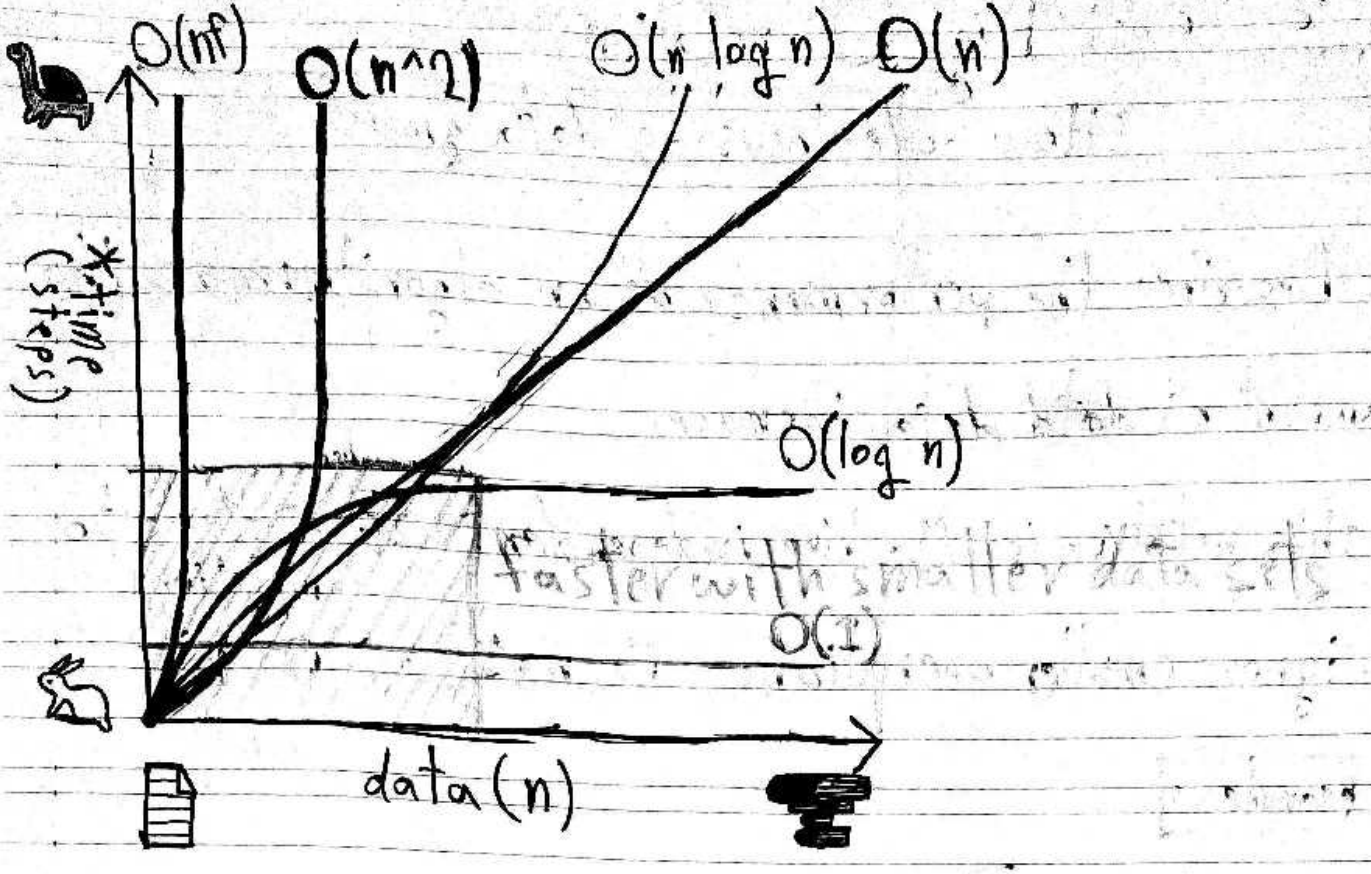
O(1) constant time

```
int addUp(int n){
  int sum = n*(n+1)/2;
  return sum;
}
```

better

3 steps O(3) → O(1)

Sabbagh



O(1): constant time
- random access of element in an array
- inserting at the beginning of a linked list

O(log n): logarithmic time
- binary search

O(n): linear time
- looping through elements in an array
- searching through a linked list

O(n log n): quasilinear time

- quick sort
- merge sort
- heap sort

O(n^2): quadratic time

- insertion sort
- selection sort
- bubble sort

O(nf): factorial time

- Travelling salesman problem

Date: / /

Subject:

Algorithms

linear search:

runtime complexity: O(n)

Disadvantages:
- Slow for large data sets

Advantages:
- fast for searches of small to medium data sets.
- Does not need to be sorted (huge advantage)
- Useful for DS that don't have random access (LinkedList)

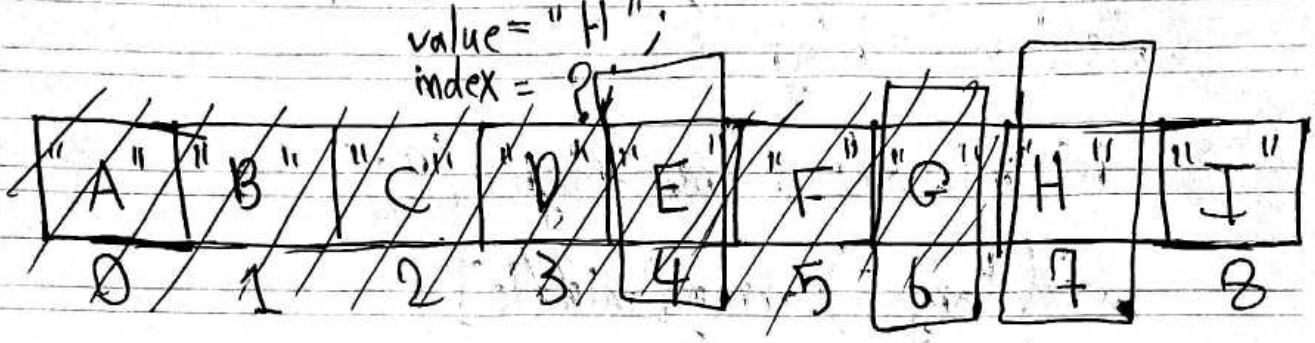
```
int linearSearch(vector<int> array, int value){
    for(int i=0; i<array.size(); i++){
        if(array[i] == value){ return i;
    }
    return -1;
}
```

Date: 1/1 (9) Subject:

binary search

Search algorithm that finds the position of a target value within a sorted array.

Half of the array is eliminated during each "step"



① since "H" is greater than "E";
we eliminate the first half of the array

② since "H" is greater than "G"
we eliminate the first half of the second half of the array

③ we return 7;

3 steps

more efficient when working with large data sets

runtime complexity: O(log n)

Sabbagh

Date: 10/8/2021

subject: binarySearch cpp function code for practice

```
int binarySearch(vector<int> array, int target){
  int low=0;
  int high = array.size() - 1;
  while(low<=high){
    int middle = (low+high)/2;
    if (target>array[middle])
      low = middle+1;
    else if (target<array[middle])
      high = middle+1;
    else
      return middle;
  }
  return -1;
}
```

low + (high - lo (target [low]) / (array[high] - array[low])

[redacted]: interpolation search

time complexity: average O(log(log(n)))
worst O(n) [value increase exponentially]

improvement over binary search, best used for "uniformly" ~~sorted arrays~~ distributed ~~arrays~~

"guesses" when a value might be based on calculated probe results, if probe is incorrect, search area is narrowed, and the new probe is :c

Date :

```
int interpolationSearch(vector<int> array, int target){
    int low = 0;
    int high = array.size()-1;

    while(target > array[low] && target < array[high] &&
    low <= high){
        int probe = low + (high-low) * (target - array[low]) /
                    (array[high] - array[low]);
        if(target < array[probe]){
            high = probe - 1;
        else if(target > array[probe])
            low = probe + 1;
        else return probe;
    } return -1;}
```

Sabbagh

Date / / 13 Subject:

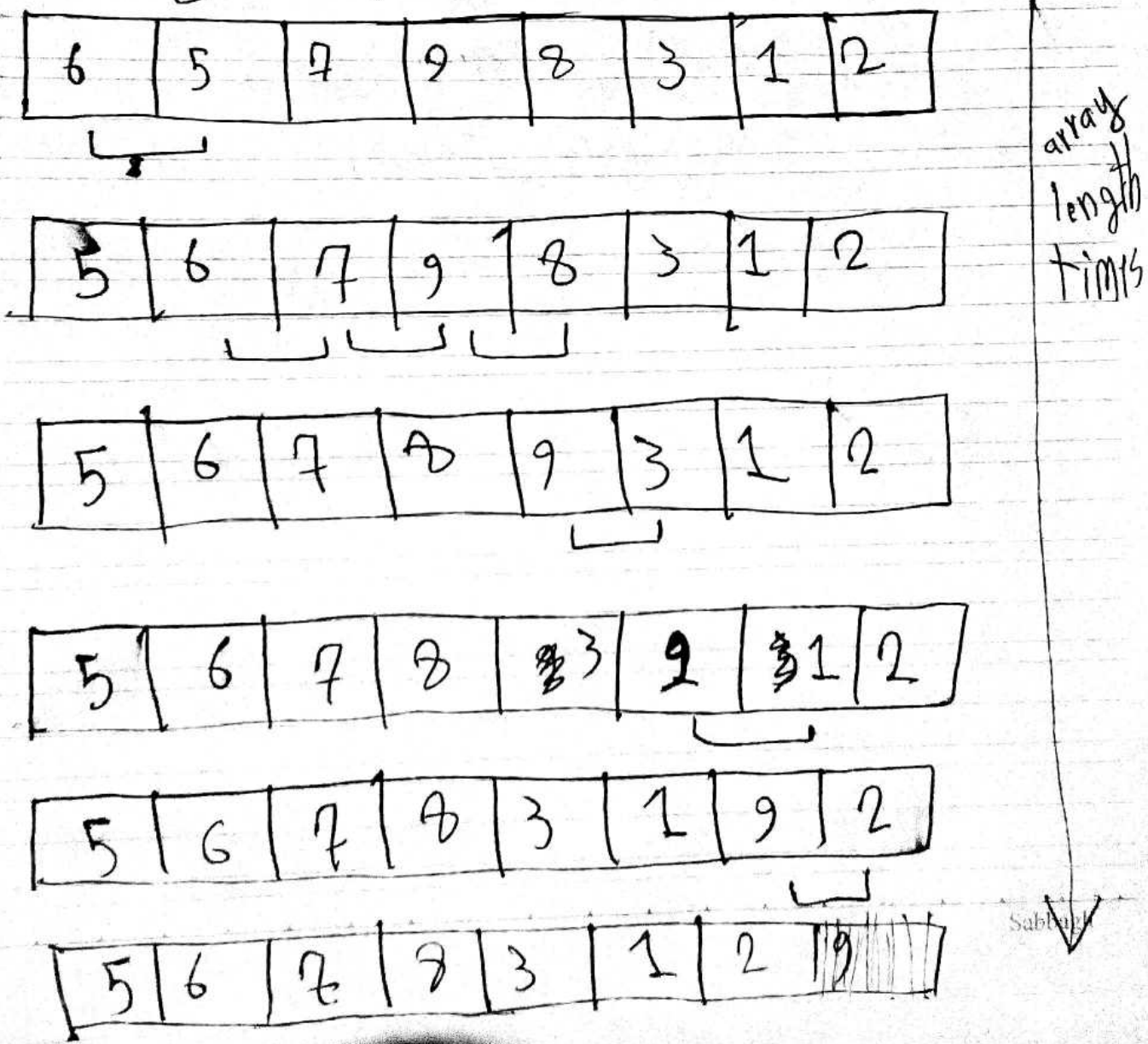
bubbleSort:

pairs of adjacent elements are compared, and the elements swapped if they are not in order.

time complexity: $O(n^2)$ Quadratic time

bad for large data sets

okay-ish for small data sets

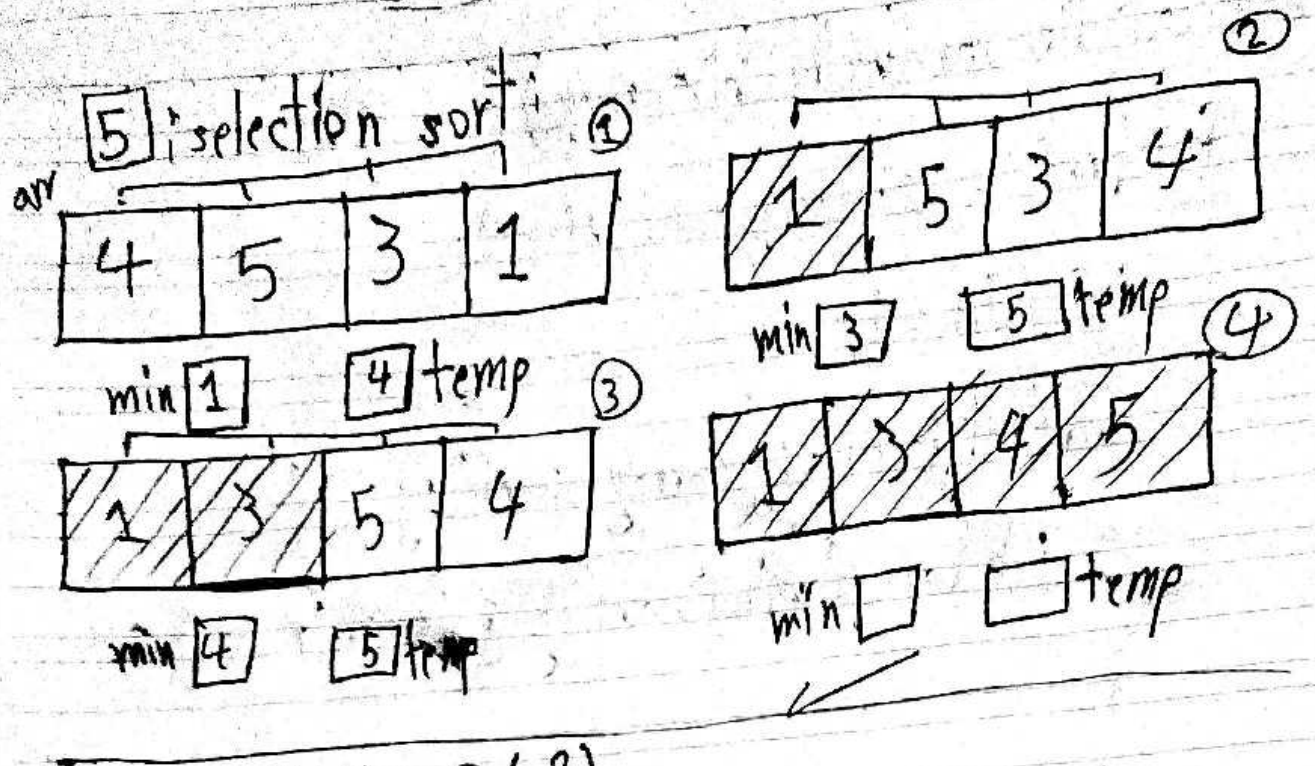


Date / /

```
void bubbleSort(vector<int>* array){
    for(int i=0; i<array.size()-1; i++)
        for(int j=0; j<array.size-i; j++){
            if(array[j] > array[j+1]){
                int temp=array[j];
                array[j]=array[j+1];
                array[j+1]=temp;
            }
        }
}
```

15

5 ; selection sort



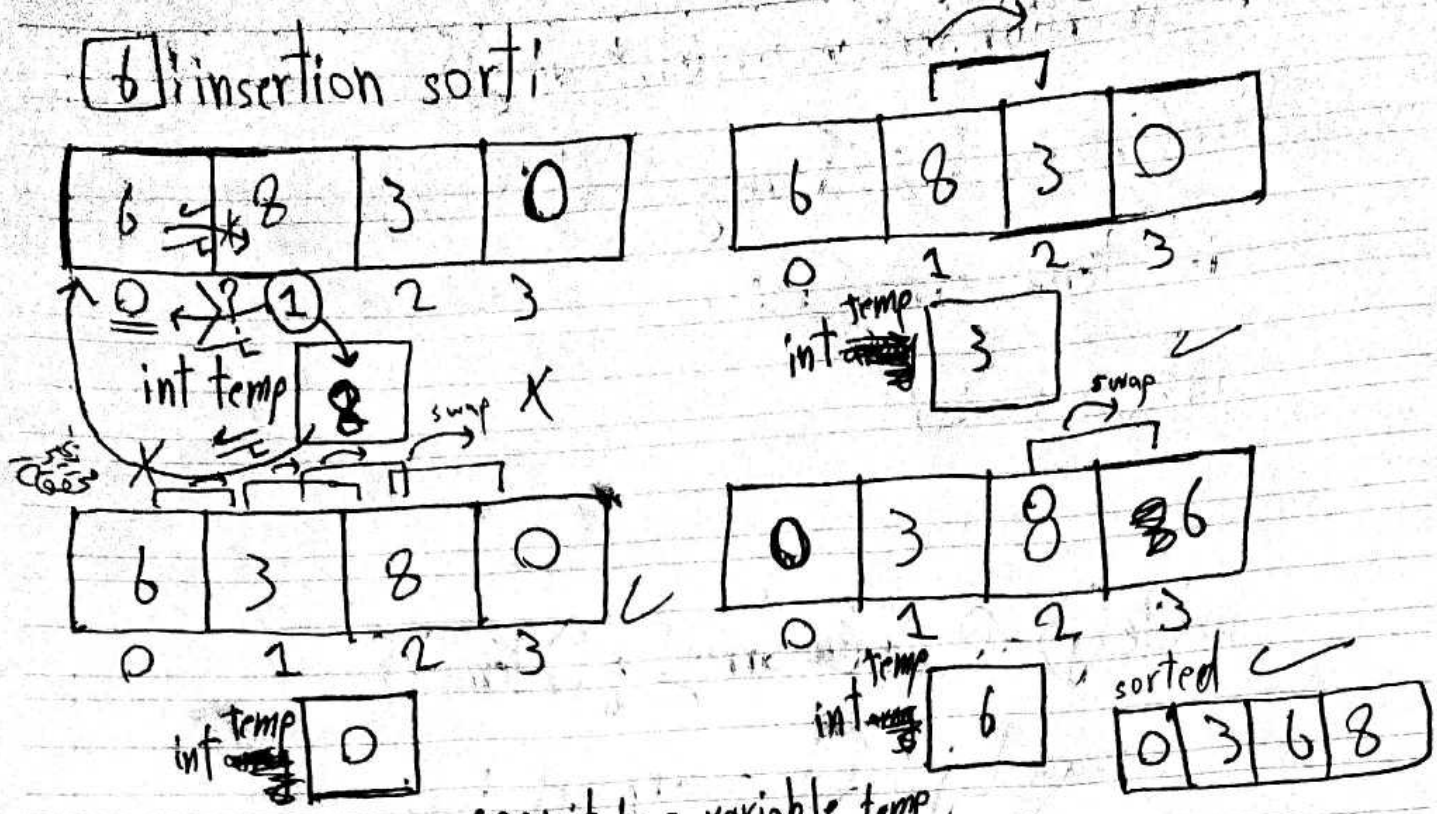
time complexity: $O(n^2)$

- okay for small data sets
  bad for large data sets

```
void selectionSort(vector<int>*array){
    for(int i=0;i<array.size();i++){
        int min=i;
        for(int j=i+1;j<array.size();j++){
            if(array[min]>array[j]) min=j;
        }
        int temp=array[i];
        array[i]=array[min];
        array[min]=temp;
    }
}
```

Date 14 , less steps than bubble sort
Best case O(n) compared to selection sort O(n²)

b) insertion sort:



copy it to a variable temp

we start at index i=1 and compare between it and the element before it, if the element before is greater we swap between them (assign before value to the element at index i=1, then we assign temp value to the before element) and repeat so on. (we check for the i-1 if true we shift i-1 and check for i-2 and so on ...)

time complexity: Quadratic time $O(n^2)$

small data sets: decent, large data sets: bad

Sabbagh

Date: 18 Subject:

```
void insertionSort(vector<int>* array){
  for(int i = 1; i<array.size(); i++){
    int temp = array[i];
    int j = i-1;

    while(j>=0 && array[j]>temp){
      array[j+1] = array[j];
      j--;
    }
    array[j+1] = temp;
  }
};
```

Date (19)

Subject

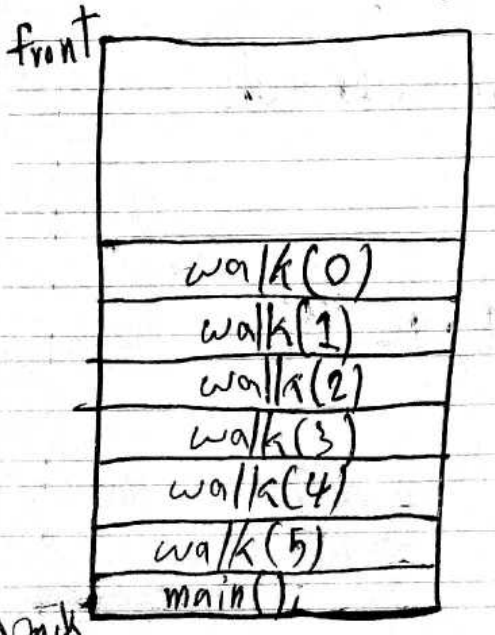
Recursion

When a thing is defined in terms of itself.

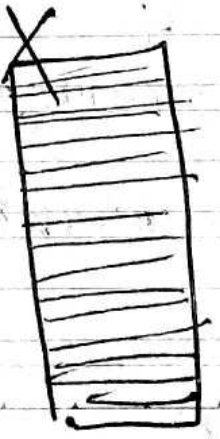
-Wikipedia-

```
void walk(int steps){
    if(steps<1) return; //base case
    std::cout<<"you made a step"<<std::endl; //action
    walk(steps-1); //recursive case
}
```

the call stack be like:



the stack overflow error when the call stack runs out of memory



Date : / / (20)

Subject

7 ↓

7 * 6

↓

7*6*5*4*3*2*1

7 x — 42
6 x — 84
5 x — 168
4 x — 336
3 x
2 x
1

recursive approach

```
int factorial(int num){
  if(num<1) return; //base case
  return num*factorial(num-1);
}
```

iterative approach

```
int factorial (int num){
  int answer = 1;
  for(int i = num; [illegible] i--){
    answer = answer * i;
  }
  return answer;
}
```

Sabbagh

(12)

```
int power(int base, int exponent){
    if (exponent<1) return; // base case
    return base * power(base, exponent - 1); // recursive case
}
```

comparsion between iteration and recursion

	iteration	recursion
implemention	uses loops	calls itself
state	control index(int steps=0)	parameter values walk(int step)
progression	moves toward value in condition	converge towards base case
termination	index satisfies condition	base case = true
size	more code, less memory	less code, more memory
speed	faster	slower

Date : / /

Subject:

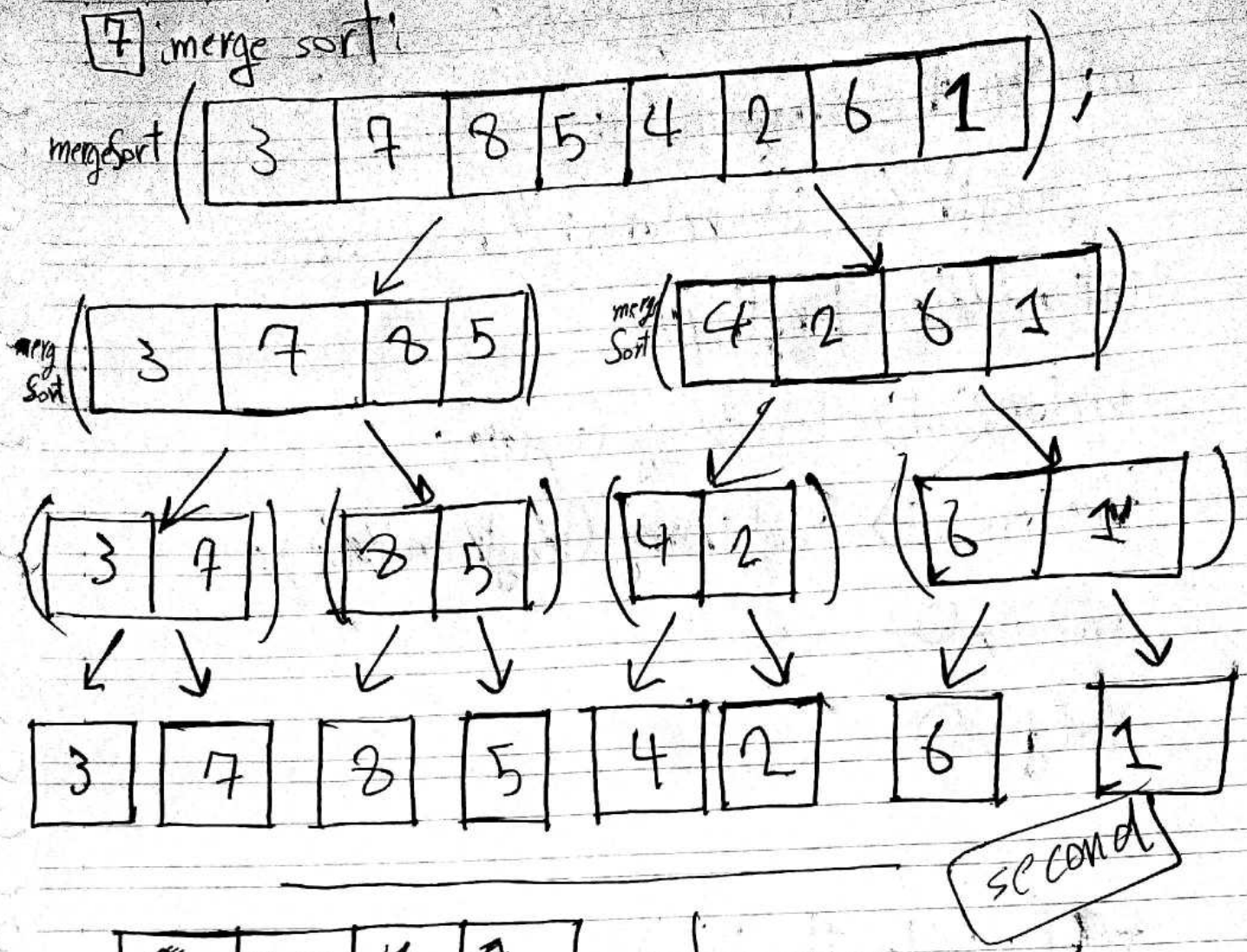
Sabbagh

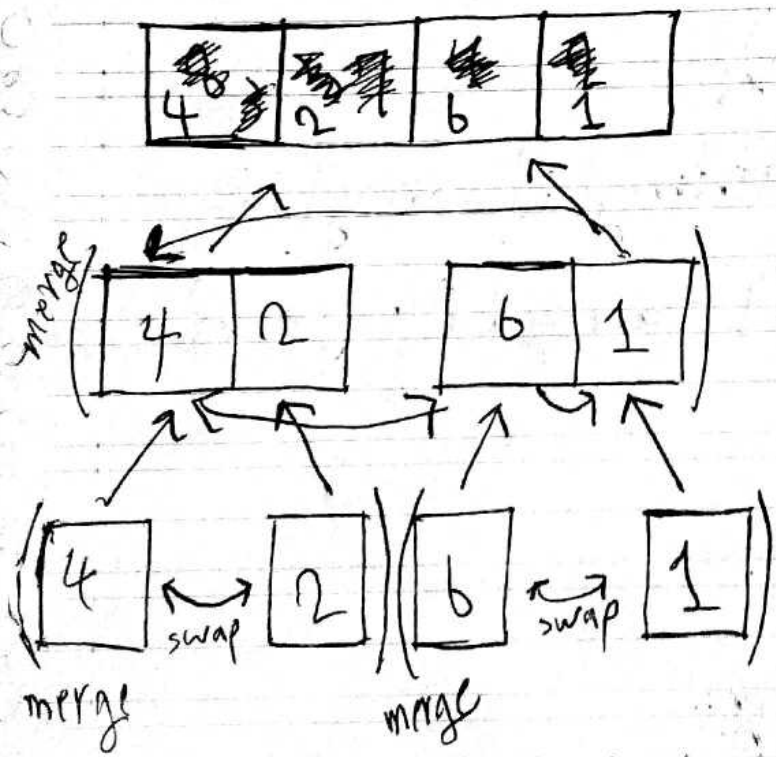
Date: / /

(23)

Subject: first

7. 'merge sort'





time complexity: $O(n \log n)$

space complexity: $O(n)$

Sabbagh

```
void mergeSort(vector<int>* array){
    int length = array.size();
    if(length <= 1) return; // base case
    int middle = length/2;
    vector<int> leftArray (middle);
    vector<int> rightArray (length-middle);
    int i=0;
    int j=0;

    for(; i<length; i++){
      if(i < middle) leftArray[i]=array[i];
      else{rightArray[j] = array[i]; j++}
    }
    mergeSort(leftArray);
    mergeSort(rightArray);
    merge(leftArray,rightArrag, array);
}
```

Sabbagh

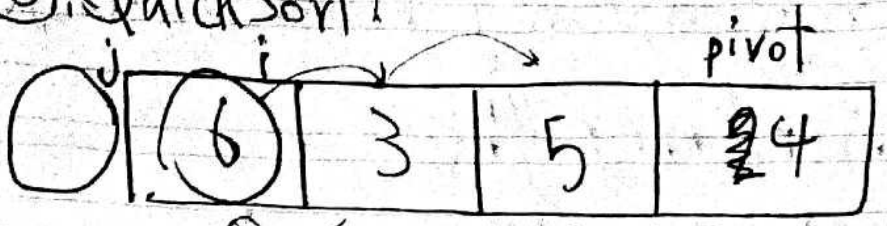
```
void merge(vector<int>* array, vector<int> leftArray, vector<int> rightArray){
    int leftSize = array.size()/2;
    int rightSize= array.length-leftSize;
    int i=0; int l=0; int r=0;

    while(l<leftSize && r<rightSize){
      if(leftArray[l]<rightArray[r]){
        array[i]= leftArray[l];
        l++; i++;
      } else {
        array[i] = rightArray[r];
        r++; i++;
      }
    while(l<leftSize){
      array[i] = leftArray[l];
      i++; l++;
    }
```

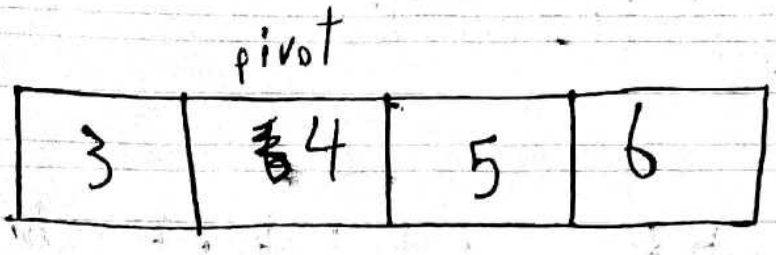
Sabbagh

```
while(r<rightSize){
    array[i] = rightArray[r];
    i++; r++;
}
}
```

8) QuickSort i



we increase i by 1 if it is less than the pivot we increase j by one then we swap between i and j we end like this:



we ended up with one step but if not we recall quickSort recursively

time complexity: Best case: O(nlogn)
avg case: O(nlogn)
worst case: $O(n^2)$ if already sorted

space complexity: O(log(n)) due to recursion

```
void quickSort(vector<int>*array, int start, int end){
    if(end<=start) return; //basecase
    int pivot = partition(array, start, end);
    quickSort(array, start, pivot-1);
    quickSort(array, pivot+1, end);
}
int partition(vector<int>*array, int start, int end){
    int pivot= array[end];
    int i= start-1;
    for(int j= start; j<=end; j++){
        if(array[j]<pivot){
            i++;
            int temp = array[i];
            array[i] =array[j];
            array[j] = temp
        }
}
```

Sabbagh

```
        i++;
        int temp = array[i];
        array[i] = array[end];
        array[end] = temp;
    }
```

Date 30 / /

Subject:

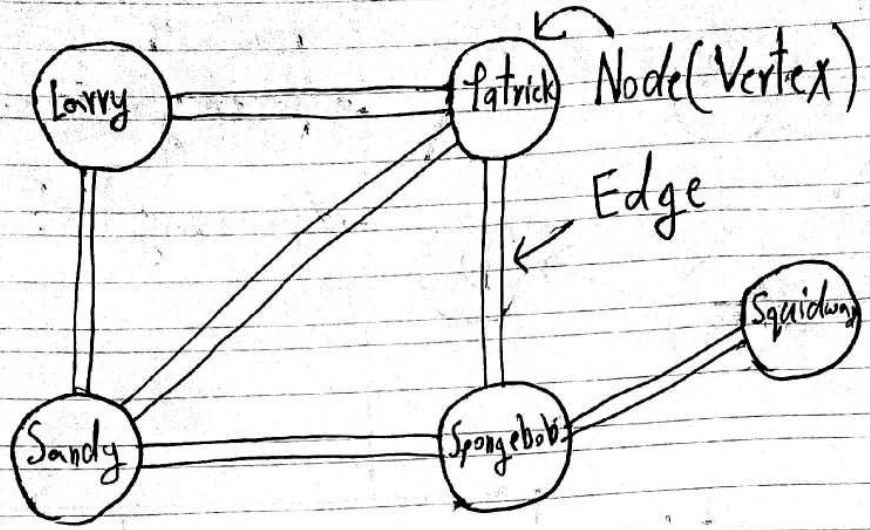
Hash Tables

Date : / / (31)

Subject:

Graphs

undirected Graph



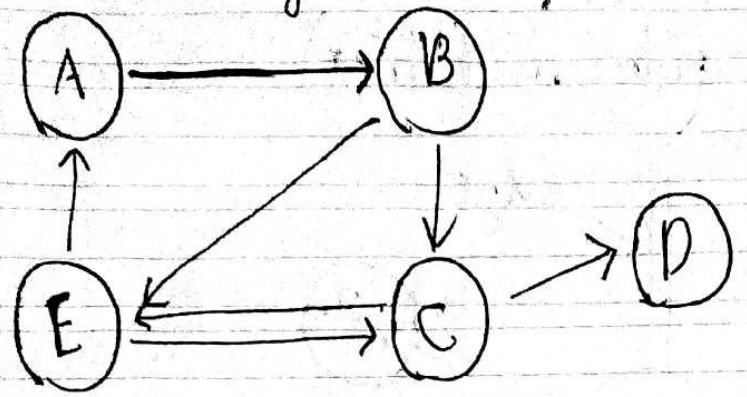
each node can represents a user if a user is friend with another then we establish an edge between them in an undirected graph since they both are friend to each other

if two nodes are connected they have what is called adjacency

Date 32 Subject:

directed graph

one way connection.



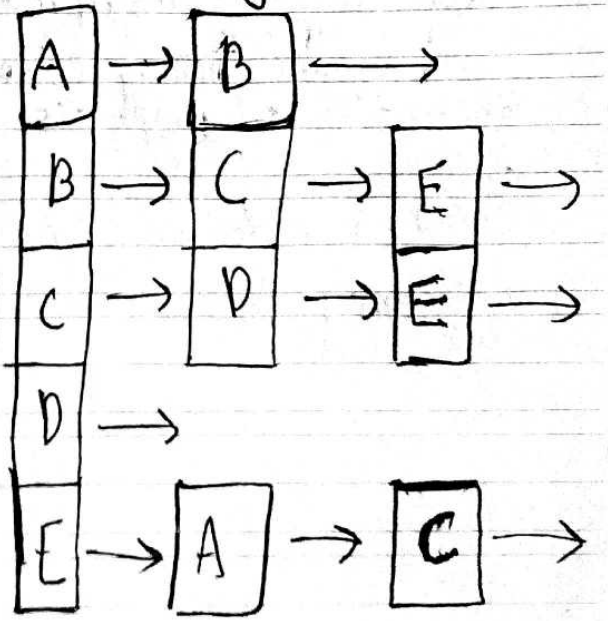
we can represents them with:
- adjacency matrix.
- adjacency list.

adjacency matrix:

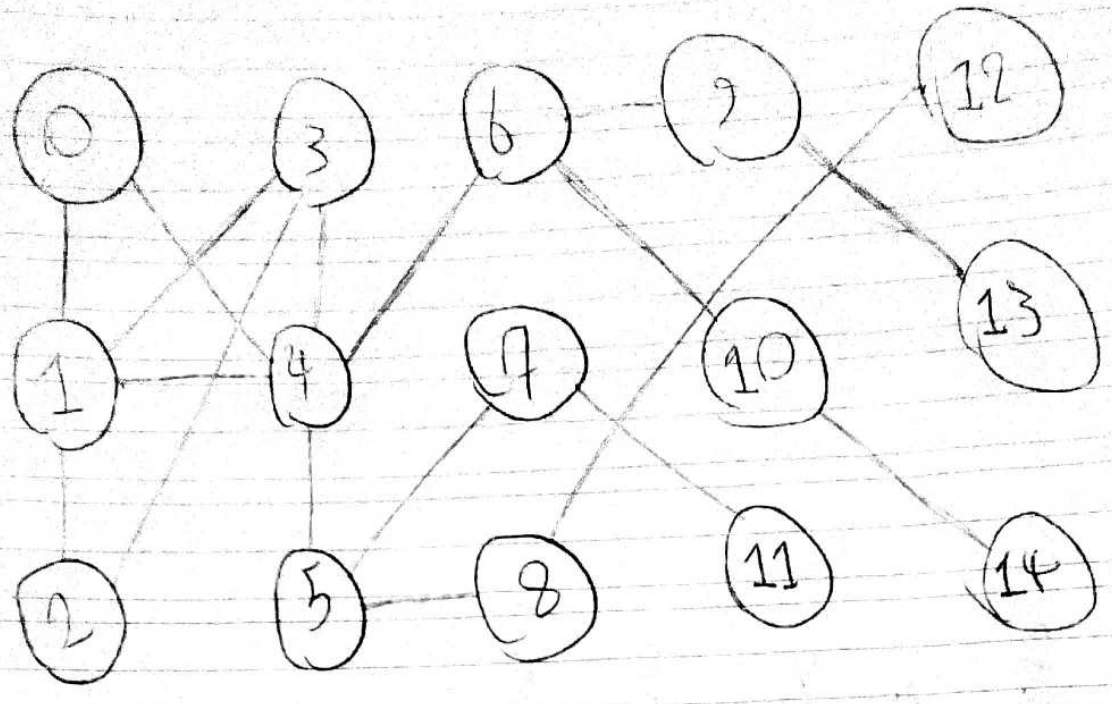
	A	B	C	D	E
A	0	1	0	0	0
B	0	0	1	0	1
C	0	0	0	1	1
D	0	0	0	0	0
E	1	0	1	0	0

time complexity: $O(1)$
space complexity: $O(V^2)$

adjacency list:



time complexity: $O(V)$
space complexity: $O(V+E)$



Date : / /

Subject.